

# Reproduction Instructions – Total Event System Solutions

---

A complete, **zero-cost, no-Lovable-required** guide to deploying this platform for a small city, town, or nonprofit. Everything below uses free tiers of well-known services. No credit card is required for the default path.

**Who this is for.** A city staffer, nonprofit coordinator, or volunteer with **no coding experience**. If you can copy/paste and click buttons, you can do this. **Time.** About 2 hours start to finish. **Cost.** \$0/month at small-city scale (see Appendix B).

---

## Part 1 – What you're building

---

You're standing up your own copy of an event-management platform that includes:

- Public event listings and free or paid ticketing
- Room and meeting-space reservations
- Vendor, sponsor, and busker (StreetBeats) applications
- Special-event permit intake
- Staff dashboards with granular permissions per department
- Email marketing campaigns + surveys
- Multi-department tenancy with per-department branding
- A built-in visual user manual at [/manual](#)

## Time + cost at a glance

| Resource                           | Free tier                        | Enough for                     |
|------------------------------------|----------------------------------|--------------------------------|
| Supabase (database, auth, storage) | 500 MB DB, 1 GB storage, 50k MAU | A town of ~10k residents       |
| Cloudflare Workers (hosting)       | 100k requests/day                | ~3k unique daily visitors      |
| Resend (email, optional)           | 3,000 emails/mo                  | ~10 campaigns/mo to 300 people |
| GitHub (code hosting)              | Unlimited public repos           | Anything                       |
| <b>Total</b>                       |                                  | <b>\$0 / month</b>             |

You can run the entire platform without spending a dollar. Paid upgrades only kick in past those limits.

## Part 2 – Install the free tools on your computer

You only do this once.

### 2.1 Git (downloads + uploads code)

- **Mac:** open Terminal, type `git --version`, press Enter. If it offers to install Command Line Tools, click **Install**.
- **Windows:** download from <https://git-scm.com/download/win>, run the installer, accept all defaults.

Verify: open a terminal and run `git --version`. You should see a version number.

### 2.2 Node.js 20 LTS (runs the website)

- Download the **LTS** installer from <https://nodejs.org/>. Run it, accept defaults.

Verify: `node -v` should print `v20.x.x` or higher.

## 2.3 Bun (installs the building blocks faster than npm)

- **Mac/Linux**, paste this in Terminal:

```
curl -fsSL https://bun.sh/install | bash
```

- **Windows (PowerShell)**:

```
powershell -c "irm bun.sh/install.ps1 | iex"
```

Close and reopen your terminal. Verify: `bun -v` should print a version number.

## 2.4 VS Code (optional – friendlier than Notepad)

- Download from <https://code.visualstudio.com/>. Used only for editing a single config file later.

## 2.5 Wrangler (uploads the site to Cloudflare)

After Node.js is installed, run:

```
npm install -g wrangler
```

Verify: `wrangler --version` prints a version.

---

## Part 3 – Get the code

---

1. Open <https://github.com/> and create a free account if you don't have one.
2. Open the project's GitHub page in your browser. Click the **Fork** button in the top-right. This makes your own copy you can modify.
3. On *your* fork, click the green **Code** button → **HTTPS** → copy the URL.
4. In your terminal, in a folder where you keep projects:

```
git clone <paste-the-url-here>
cd <project-folder-name>
bun install
```

The last step downloads ~500 MB of building blocks. Grab a coffee.

## Part 4 – Create your free database (Supabase)

Supabase gives you a Postgres database, user accounts, file storage, and security policies in one free package. No credit card.

### 4.1 Create the project

1. Go to <https://supabase.com> → **Start your project** → sign in with GitHub.
2. Click **New Project**.
  - Name: anything (e.g. `myCity-events` )
  - Database password: **write this down somewhere safe** — you can't recover it.
  - Region: pick the one closest to your residents.
  - Plan: **Free**.
3. Wait ~2 minutes for it to provision.

### 4.2 Grab your keys

Open **Project Settings** → **API** and copy these three values into a temporary text file:

| Value                                                                                                     | What you'll do with it                                                   |
|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Project URL</b><br>( <code>https://xxxx.supabase.co</code> )                                           | Goes into the code in step 4.3                                           |
| <b>publishable</b> / <b>anon</b> <b>key</b><br>( <code>sb_publishable_...</code> or <code>eyJ...</code> ) | Goes into the code in step 4.3                                           |
| <b>service_role</b> <b>key</b> (a long secret)                                                            | Goes into your secrets later (Part 8). <b>Never share this publicly.</b> |

### 4.3 Paste your URL + publishable key into the code

This is the only file edit a non-coder has to make.

Open `src/integrations/supabase/config.ts` in VS Code and replace the two lines:

```
export const SUPABASE_URL = "https://YOUR-PROJECT-REF.supabase.co";
export const SUPABASE_PUBLISHABLE_KEY = "sb_publishable_...your-key...";
```

Save the file. (The publishable key is safe in code — row-level security protects every table.)

### 4.4 Run the database migrations

The repo has 38 numbered SQL files under `supabase-migrations/` that build all the tables.

**Easy way (Supabase dashboard):**

1. In Supabase, open **SQL Editor** → **New query**.
2. Open `supabase-migrations/001_staff_portal.sql` from your code folder in VS Code.
3. Copy the entire file. Paste into the SQL Editor. Click **Run**. Wait for "Success."
4. Repeat for every file in numerical order through `038_communications_surveys.sql`. (Yes, all 38. It takes ~15 minutes.)

**Faster way (one terminal command):** If you have `psql` installed (Mac: `brew install libpq`; Windows: comes with the Postgres installer), copy the connection string from **Settings** → **Database** → **Connection string** → **URI**, then:

```
export DATABASE_URL="postgresql://postgres:YOUR-PASSWORD@db.YOUR-REF.supabase.co:5432/postgres"
for f in supabase-migrations/*.sql; do
  echo ">>> $f"
  psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f "$f"
done
```

### 4.5 Turn on the two database extensions

In Supabase → **Database** → **Extensions**, search for and enable:

- `pg_cron` (lets the database run scheduled jobs — for email dispatch)
- `pg_net` (lets the database make HTTP calls — used by the cron job)

Both are one-click toggles.

---

## Part 5 – Run it on your laptop first

---

Before deploying anywhere, make sure it works locally.

```
bun dev
```

Open `http://localhost:8080` in your browser. You should see the home page.

### 5.1 Create your first admin user

1. Click **Sign up** on the site, create an account with your email.
2. Open Supabase → **Authentication** → **Users**. Copy your user ID (a long UUID).
3. Back in Supabase → **SQL Editor**, run (paste your ID):

```
insert into public.user_roles (user_id, role)
values ('paste-your-user-id-here', 'admin');
```

4. Refresh the site. The **Event Ops** sidebar appears.
5. Open `/staff/admin/departments` and create your first department (e.g. "Parks & Rec").
6. Open `/staff/admin/permissions` and give yourself a role in that department.

## 5.2 Common local errors

| Message                                    | Fix                                                                                 |
|--------------------------------------------|-------------------------------------------------------------------------------------|
| Port 8080 already in use                   | Close whatever's using it, or run <code>PORT=8081 bun dev</code> .                  |
| Failed to fetch on every page              | Your Supabase URL or key in <code>config.ts</code> is wrong. Double-check Part 4.3. |
| new row violates row-level security policy | A migration was skipped. Re-run them all from 001.                                  |

## Part 6 – Pick a payment option (or skip)

You have five paths. Pick **one** based on your situation.

### Option A – No payments (recommended if all your events are free)

**Do nothing.** The platform works fine without payments. Ticketing pages will show "Free" and skip checkout. **Skip ahead to Part 7.**

### Option B – USAePay (already wired up – best for U.S. municipalities)

USAePay is built into the codebase ( `src/lib/payments.functions.ts` + `src/lib/usaepay.server.ts` ). It uses your existing merchant bank account — typically cheaper per transaction than Stripe/PayPal and often already approved for government use.

1. Apply for a USAePay merchant account at <https://usaepay.com/>. Many cities already have one through their bank.
2. Log into the USAePay merchant console. Under **Settings** → **API Keys**, generate an API Key and an API PIN.
3. You'll add three secrets in Part 8:
  - `USAEPAY_API_KEY` — the key from above
  - `USAEPAY_API_PIN` — the PIN from above

- `USAEPAY_MODE` — set to `sandbox` while testing, then change to `live`
4. Test with USAePay's sandbox test card `4000100011112224`, any future expiry, CVV `123`.

Skip ahead to **Part 7**.

## Option C – Stripe (easiest signup, 2.9% + 30¢ per transaction)

Stripe is not wired up by default — you add ~30 lines of code. If you can copy/paste, you can do this.

1. Create a free Stripe account at <https://stripe.com>.
2. Under **Deve**lopers → **API keys**, copy your **Secret key** (`sk_test_...`).
3. Under **Deve**lopers → **Webhooks**, click **Add endpoint**. URL: `https://YOUR-DOMAIN/api/public/stripe-webhook`. Select events: `checkout.session.completed`, `payment_intent.succeeded`, `payment_intent.payment_failed`. Save and copy the **Signing secret** (`whsec_...`).
4. In your code folder, install the Stripe SDK:

```
bun add stripe
```

5. Create `src/lib/stripe.server.ts` with this content:



```

// SERVER ONLY – Stripe helpers. Never import from client code.
import Stripe from "stripe";

export function loadStripe(): Stripe | null {
  const key = process.env.STRIPE_SECRET_KEY;
  if (!key) return null;
  return new Stripe(key, { apiVersion: "2024-06-20" });
}

export async function createCheckoutSession(opts: {
  amountCents: number;
  currency: string;
  successUrl: string;
  cancelUrl: string;
  metadata?: Record<string, string>;
  description?: string;
}) {
  const stripe = loadStripe();
  if (!stripe) throw new Error("STRIPE_SECRET_KEY not configured");
  return stripe.checkout.sessions.create({
    mode: "payment",
    line_items: [{
      price_data: {
        currency: opts.currency,
        product_data: { name: opts.description ?? "Ticket" },
        unit_amount: opts.amountCents,
      },
      quantity: 1,
    }],
    success_url: opts.successUrl,
    cancel_url: opts.cancelUrl,
    metadata: opts.metadata,
  });
}

```

6. In `src/lib/payments.functions.ts`, find the `charge` handler. Where it calls `loadUsaepayConfig()`, branch on `process.env.PAYMENT_PROVIDER`:

```
if (process.env.PAYMENT_PROVIDER === "stripe") {  
  const { createCheckoutSession } = await import("./stripe.server");  
  const session = await createCheckoutSession({ /* ...fields... */ });  
  return { provider: "stripe", redirectUrl: session.url! };  
}  
  
// existing USAePay path stays below
```

7. Add these secrets in Part 8: `STRIPE_SECRET_KEY` ,  
`STRIPE_WEBHOOK_SECRET` , `PAYMENT_PROVIDER=stripe` .

Skip ahead to **Part 7**.

### Option D – PayPal (no monthly fee, 3.49% + 49¢)

1. Create a PayPal Business account → <https://developer.paypal.com>.
2. In the developer dashboard, **Apps & Credentials** → **Create App**. Copy the **Client ID** and **Secret**.
3. Install the SDK:

```
bun add @paypal/checkout-server-sdk
```

4. Create `src/lib/paypal.server.ts` :

```
import * as paypal from "@paypal/checkout-server-sdk";

function client() {
  const id = process.env.PAYPAL_CLIENT_ID!;
  const secret = process.env.PAYPAL_CLIENT_SECRET!;
  const env = process.env.PAYPAL_MODE === "live"
    ? new paypal.core.LiveEnvironment(id, secret)
    : new paypal.core.SandboxEnvironment(id, secret);
  return new paypal.core.PayPalHttpClient(env);
}

export async function createOrder(amountCents: number, currency =
  "USD") {
  const req = new paypal.orders.OrdersCreateRequest();
  req.requestBody({
    intent: "CAPTURE",
    purchase_units: [{
      amount: { currency_code: currency, value: (amountCents /
        100).toFixed(2) },
    }],
  });
  const res = await client().execute(req);
  return res.result;
}
```

5. Branch in `payments.functions.ts` the same way as Stripe (Option C, step 6) but on `PAYMENT_PROVIDER === "paypal"`.
6. Add secrets: `PAYPAL_CLIENT_ID`, `PAYPAL_CLIENT_SECRET`, `PAYPAL_MODE=sandbox` (or `live`), `PAYMENT_PROVIDER=paypal`.

## Option E – Square (good for cities that already use Square for in-person POS)

1. Create a Square account → <https://developer.squareup.com>.
2. **Applications** → + → name it. Copy the **Access Token** and **Application ID** from the **Sandbox** tab (switch to **Production** when ready).
3. Install: `bun add square`
4. Create `src/lib/square.server.ts` :

```
import { Client, Environment } from "square";

export function loadSquare() {
  const token = process.env.SQUARE_ACCESS_TOKEN;
  if (!token) return null;
  return new Client({
    accessToken: token,
    environment: process.env.SQUARE_MODE === "live"
      ? Environment.Production
      : Environment.Sandbox,
  });
}
```

5. Use Square's `paymentsApi.createPayment` with the source nonce from the Square Web Payments SDK on the front-end. Wire into `payments.functions.ts` the same way.
6. Add secrets: `SQUARE_ACCESS_TOKEN` , `SQUARE_APPLICATION_ID` , `SQUARE_MODE` , `PAYMENT_PROVIDER=square` .

---

## Part 7 – Optional add-on services

---

Each of these is **optional**. Skip any you don't need — the platform works without them.

### 7.1 Email campaigns (Resend, free tier 3,000/mo)

Required only if you want to send email blasts from `/staff/communications` .

1. Sign up at <https://resend.com>.
2. **Domains** → add yours. Resend gives you DNS records to add at your domain registrar (GoDaddy, Namecheap, Cloudflare DNS, etc.).
3. **API Keys** → create one. Copy it.
4. Add secrets in Part 8: `RESEND_API_KEY` , `RESEND_FROM` (e.g. `City Events <hello@yourcity.gov>` ), `DISPATCH_SECRET` (any random 32-character string you make up).
5. After deploying, schedule the database to ping the dispatch endpoint every minute. In Supabase → **SQL Editor**:

```

select cron.schedule(
  'dispatch-due-campaigns',
  '* * * * *',
  $$select net.http_post(
    url := 'https://YOUR-DOMAIN/api/public/dispatch-due',
    headers := jsonb_build_object('x-dispatch-secret',
      'YOUR_DISPATCH_SECRET')
  )$$
);

```

## 7.2 Auto-generated images (optional, ~\$0.04/image)

Used to auto-create event hero images. The platform works fine without this — staff can upload images manually.

If you want it: open `src/lib/auto-image.server.ts`. Replace the function body to call one of:

- **OpenAI** (best quality, \$0.04/image): `POST https://api.openai.com/v1/images/generations` with header `Authorization: Bearer $OPENAI_API_KEY`.
- **Google Gemini** (cheaper, similar quality): `POST https://generativelanguage.googleapis.com/v1beta/models/imgen-3.0:generateImages?key=$GOOGLE_AI_API_KEY`.
- **Stability AI** (open weights): `POST https://api.stability.ai/v2beta/stable-image/generate/core`.

Keep the function signature the same: take a prompt → return a PNG buffer → upload to the `auto-images` Supabase storage bucket → return the public URL. Add whichever provider's key as a secret.

## 7.3 Google sign-in (optional)

1. Supabase → **Authentication** → **Providers** → enable **Google**.
2. Google Cloud Console → **APIs & Services** → **Credentials** → **Create OAuth client ID** → Web application.
3. Authorized redirect URI: `https://YOUR-REF.supabase.co/auth/v1/callback`.
4. Paste the Client ID + Secret into Supabase. Save.

## 7.4 Social Command Center (optional – schedule Facebook/Instagram/LinkedIn posts)

1. Create Meta and LinkedIn developer apps at <https://developers.facebook.com> and <https://developer.linkedin.com>.
2. OAuth callbacks:
  - `https://YOUR-DOMAIN/api/public/oauth/meta/callback`
  - `https://YOUR-DOMAIN/api/public/oauth/linkedin/callback`
3. After deploy, paste the App IDs/secrets at `/staff/admin/social-integrations`.

## 7.5 Custom domain (optional, ~\$12/yr)

Buy a domain at Cloudflare Registrar (cheapest, at-cost) or Namecheap. Skip until Part 8 is done — easier to point at the Worker once it exists.

---

# Part 8 – Put it on the internet (free Cloudflare Workers)

---

Cloudflare Workers gives you 100,000 requests/day free, no credit card.

## 8.1 Create the Cloudflare account

- Sign up at <https://cloudflare.com>.
- No credit card required for the Workers free plan.

## 8.2 Log in from your terminal

```
wrangler login
```

A browser opens; click **Allow**.

## 8.3 Push your secrets

For each secret you collected above, run `wrangler secret put NAME` — it prompts for the value (which is hidden as you paste).

**Required for every install:**

```
wrangler secret put SUPABASE_URL
wrangler secret put SUPABASE_PUBLISHABLE_KEY
wrangler secret put EXT_SUPABASE_SERVICE_ROLE_KEY
wrangler secret put SITE_URL           # e.g. https://your-
                                         worker.workers.dev
```

### If you set up email (7.1):

```
wrangler secret put RESEND_API_KEY
wrangler secret put RESEND_FROM
wrangler secret put DISPATCH_SECRET
```

### If you set up payments – choose the one you picked:

```
# USAePay
wrangler secret put USAEPAY_API_KEY
wrangler secret put USAEPAY_API_PIN
wrangler secret put USAEPAY_MODE

# Stripe
wrangler secret put STRIPE_SECRET_KEY
wrangler secret put STRIPE_WEBHOOK_SECRET
wrangler secret put PAYMENT_PROVIDER    # value: stripe

# PayPal
wrangler secret put PAYPAL_CLIENT_ID
wrangler secret put PAYPAL_CLIENT_SECRET
wrangler secret put PAYPAL_MODE
wrangler secret put PAYMENT_PROVIDER    # value: paypal

# Square
wrangler secret put SQUARE_ACCESS_TOKEN
wrangler secret put SQUARE_APPLICATION_ID
wrangler secret put SQUARE_MODE
wrangler secret put PAYMENT_PROVIDER    # value: square
```

## 8.4 Build and deploy

```
bun run build
wrangler deploy
```

Wrangler prints a URL like `https://total-event-system.YOUR-NAME.workers.dev`. Open it. You should see your home page **live on the internet**.

## 8.5 Update SITE\_URL

Once you have the live URL, re-run `wrangler secret put SITE_URL` with that URL and `wrangler deploy` again. (Unsubscribe links and OAuth callbacks need the real URL.)

## 8.6 Attach a custom domain (optional)

1. In Cloudflare, add your domain (move nameservers — they walk you through it).
2. Open your Worker → **Settings** → **Triggers** → **Add Custom Domain** → enter `events.yourcity.gov`. Done.

## 8.7 Other free hosts

If you'd rather not use Cloudflare, these all work too. You'll need to remove `@cloudflare/vite-plugin` from `vite.config.ts` and follow the host's TanStack Start adapter docs:

- **Vercel** (free hobby tier): <https://vercel.com>
- **Netlify** (free starter): <https://netlify.com>
- **Render** (free web service tier): <https://render.com>
- **Fly.io** (free allowance): <https://fly.io>

---

## Part 9 – Smoke test (verify everything works)

---

Click each of these on your live URL. Each one should load without errors.



| URL                                      | What you should see                                                                      |
|------------------------------------------|------------------------------------------------------------------------------------------|
| <a href="#">/</a>                        | Home page (CMS-driven; bare until you add content at <a href="#">/staff/admin/home</a> ) |
| <a href="#">/events</a>                  | Public event list (empty until you create events)                                        |
| <a href="#">/manual</a>                  | Full visual user manual                                                                  |
| <a href="#">/staff</a>                   | Staff dashboard (sign in first)                                                          |
| <a href="#">/staff/admin/permissions</a> | Admin permissions matrix                                                                 |
| <a href="#">/staff/admin/departments</a> | Create/edit departments                                                                  |
| <a href="#">/staff/communications</a>    | Email campaigns (if Resend is set up)                                                    |
| <a href="#">/staff/surveys</a>           | Surveys list                                                                             |
| <a href="#">/survey/&lt;id&gt;</a>       | Public anonymous survey form (after creating one)                                        |

## Part 10 – Day-to-day operations

### Update the code

When the upstream repo gets new features:

```
git pull
bun install
bun run build
wrangler deploy
```

If new files appear in [supabase-migrations/](#) , run those in Supabase SQL Editor first.

### Back up your data

Supabase free tier keeps 7 days of point-in-time recovery automatically. For longer backups: **Database** → **Backups** → **Download** weekly.

### Invite more admins

Sign them up on the live site, then run:

```
insert into public.user_roles (user_id, role)
values ('<their-user-id>', 'admin');
```

## Where to look when something breaks

- **Live site logs:** `wrangler tail` in your terminal — streams every request and error.
- **Database logs:** Supabase → **Logs** → **Postgres logs**.
- **Email send failures:** Resend dashboard → **Emails** tab.

## Where to get help

- This repo's **Issues** tab on GitHub.
  - Supabase Discord: <https://discord.supabase.com>
  - Cloudflare community: <https://community.cloudflare.com>
  - TanStack Start docs: <https://tanstack.com/start>
-

## Part 11 – Troubleshooting

| Error you see                                       | What it means                                                                           | Fix                                                                                                                                                                       |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Failed to resolve import                            | A file the code expects is missing.                                                     | Re-run <code>bun install</code> . Make sure you didn't delete any files in <code>src/</code> .                                                                            |
| new row violates row-level security policy          | You skipped a migration.                                                                | Re-run all migrations in <code>supabase-migrations/</code> in order.                                                                                                      |
| Campaign stuck in <code>scheduled</code> forever    | The <code>pg_cron</code> job isn't running.                                             | Run <code>select * from cron.job;</code> in Supabase to confirm the schedule exists. Make sure <code>DISPATCH_SECRET</code> matches between your secret and the cron SQL. |
| Emails come from <code>onboarding@resend.dev</code> | <code>RESEND_FROM</code> isn't set, so Resend uses its test sender (capped at 100/day). | <code>wrangler secret put RESEND_FROM</code> with a verified domain.                                                                                                      |
| <code>__dirname</code> is not defined at runtime    | A Node-only npm package slipped in.                                                     | Find and replace it with a Web-standard / fetch-based alternative — Cloudflare Workers can't shim <code>__dirname</code> .                                                |
| Module not found: <code>cloudflare:workers</code>   | You're running on plain Node, not Wrangler.                                             | Use <code>wrangler dev</code> , or switch hosts and remove <code>@cloudflare/vite-plugin</code> from <code>vite.config.ts</code> .                                        |
| <code>USAEPAY_API_KEY</code> not configured         | You picked USAePay but didn't add the secrets.                                          | <code>wrangler secret put USAEPAY_API_KEY</code> etc.                                                                                                                     |
| Stripe webhook signature mismatch                   | The signing secret you copied is for a different endpoint.                              | In Stripe → Webhooks, click your endpoint → re-copy the signing secret → <code>wrangler secret put STRIPE_WEBHOOK_SECRET</code> .                                         |

| Error you see                                         | What it means                                                     | Fix                                                                                                                                           |
|-------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Google sign-in says "redirect URI mismatch"           | The URI in Google Cloud Console doesn't exactly match Supabase's. | Copy it from Supabase verbatim — including <code>https://</code> and the trailing path.                                                       |
| Wrangler says "You need to login"                     | Token expired.                                                    | <code>wrangler login</code> again.                                                                                                            |
| Worker deploy fails — "size limit exceeded"           | Bundle too big (rare).                                            | Make sure you ran <code>bun run build</code> , not <code>bun build</code> . The Vite build tree-shakes; a raw <code>bun build</code> doesn't. |
| Local dev fails — <code>bun: command not found</code> | Bun isn't in your PATH yet.                                       | Close and reopen your terminal. On Windows, also restart VS Code.                                                                             |

---

## Appendix A – Full environment

## variable reference

| Name                                                                                               | Where to set it                  | Required?                     | Why?                                    |
|----------------------------------------------------------------------------------------------------|----------------------------------|-------------------------------|-----------------------------------------|
| <code>SUPABASE_URL</code>                                                                          | <code>wrangler secret put</code> | Yes                           | You                                     |
| <code>SUPABASE_PUBLISHABLE_KEY</code>                                                              | <code>wrangler secret put</code> | Yes                           | Sup key                                 |
| <code>EXT_SUPABASE_SERVICE_ROLE_KEY</code>                                                         | <code>wrangler secret put</code> | Yes                           | Sup (sei exp                            |
| <code>SITE_URL</code>                                                                              | <code>wrangler secret put</code> | Yes                           | You <code>htt</code>                    |
| <code>RESEND_API_KEY</code>                                                                        | <code>wrangler secret put</code> | If using email                | Res                                     |
| <code>RESEND_FROM</code>                                                                           | <code>wrangler secret put</code> | If using email                | <code>Dis</code><br><code>&lt;yc</code> |
| <code>DISPATCH_SECRET</code>                                                                       | <code>wrangler secret put</code> | If using scheduled email      | Rar sha                                 |
| <code>USAEPAY_API_KEY</code> /<br><code>USAEPAY_API_PIN</code> /<br><code>USAEPAY_MODE</code>      | <code>wrangler secret put</code> | If using USAePay              | Mer<br>= :                              |
| <code>PAYMENT_PROVIDER</code>                                                                      | <code>wrangler secret put</code> | If using Stripe/PayPal/Square | One<br><code>sqi</code>                 |
| <code>STRIPE_SECRET_KEY</code> /<br><code>STRIPE_WEBHOOK_SECRET</code>                             | <code>wrangler secret put</code> | If using Stripe               | Fro                                     |
| <code>PAYPAL_CLIENT_ID</code> /<br><code>PAYPAL_CLIENT_SECRET</code> /<br><code>PAYPAL_MODE</code> | <code>wrangler secret put</code> | If using PayPal               | Fro<br>das                              |

| Name                                                                                             | Where to set it     | Required?                      | Why?                 |
|--------------------------------------------------------------------------------------------------|---------------------|--------------------------------|----------------------|
| <code>SQUARE_ACCESS_TOKEN</code> / <code>SQUARE_APPLICATION_ID</code> / <code>SQUARE_MODE</code> | wrangler secret put | If using Square                | For donations        |
| <code>OPENAI_API_KEY</code> / <code>GOOGLE_AI_API_KEY</code> / <code>STABILITY_API_KEY</code>    | wrangler secret put | If using auto-generated images | Pictures of the town |

For local development, put the same values in a `.dev.vars` file at the repo root (one `KEY=value` per line). `.dev.vars` is gitignored.

## Appendix B – Cost calculator

| Scale                                                 | Supabase | Cloudflare Workers | Resend   | Total          |
|-------------------------------------------------------|----------|--------------------|----------|----------------|
| Small town (100 active users, 1k visits/mo)           | Free     | Free               | Free     | <b>\$0/mo</b>  |
| Mid-size city (1k users, 30k visits/mo, weekly email) | Free     | Free               | Free     | <b>\$0/mo</b>  |
| Large city (10k users, 300k visits/mo, daily email)   | Pro \$25 | Free               | Pro \$20 | <b>\$45/mo</b> |

Stripe/PayPal/Square fees are per-transaction (no monthly fee). USAePay typically has a small monthly gateway fee but lower per-transaction rates — usually a win once you process more than a few thousand dollars/mo.

## Appendix C – Glossary

- **RLS (Row-Level Security)**: Database rules that say "this user can only see their own rows."
- **Environment variable** / `.dev.vars`: A file holding secrets for *local* development. Never committed to GitHub.
- **Secret**: Same idea, but stored on the host (Cloudflare) for production.

- **Migration:** A numbered SQL file that adds or changes tables. You run them in order.
  - **Worker:** A small program Cloudflare runs at the edge — handles incoming web requests for you.
  - **SSR:** Server-Side Rendering — the page is built on the server before being sent to the browser.
  - **OAuth:** The "Sign in with Google" flow.
  - **Webhook:** A URL on your site that another service (Stripe, Resend) calls when something happens.
  - **pg\_cron** : A Postgres extension that runs SQL on a schedule (like a cron job inside the database).
- 

**You now own every layer:** code (GitHub), database (Supabase), runtime (Cloudflare), payments (your choice). No vendor lock-in, no monthly bill at small scale, no Lovable required.